AI DOCUMENT SUMMARIZER

An Intelligent Extractive, Abstractive and Hybrid Summarization Platform for Long-Form Documents

---

A Minor Project Report submitted in partial fulfilment of the requirements for the degree of

Master of Computer Applications (MCA)

Submitted by:

Name	Roll Number
Vinay Guleria	25MCI10004
Anish Sharma	25MCI10065
Rishav Thakur	25MCI10008

Department of Computer Applications

Academic Year 2025 – 2026

CERTIFICATE

This is to certify that the minor project entitled **"AI Document Summarizer"** submitted by **Vinay Guleria (25MCI10004)**, **Anish Sharma (25MCI10065)** and **Rishav Thakur (25MCI10008)** in partial fulfilment of the requirements for the award of the degree of **Master of Computer Applications** is a record of bonafide work carried out by them under my supervision and guidance.

The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma. The software developed as part of this project has been examined and found satisfactory.

Project Guide _____

Head of Department _____

External Examiner _____

Date: _____

Place: _____

DECLARATION

We hereby declare that the minor project report entitled **"AI Document Summarizer"** is our own original work and has been carried out by us. This work has not been submitted, either in part or in full, to any other university or institution for the award of any degree or diploma. All sources of information, libraries, models and reference material used in this project have been duly acknowledged in the References section of this report.

Name	Roll Number	Signature
Vinay Guleria	25MCI10004	_____
Anish Sharma	25MCI10065	_____
Rishav Thakur	25MCI10008	_____

Date: _____

Place: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide for the continuous encouragement, technical insight and constructive criticism provided throughout the development of this project. Their guidance helped us translate an initial idea into a working full-stack application.

We are grateful to the Department of Computer Applications for providing the laboratory facilities, software environment and academic support needed to complete this work. We also thank the Head of the Department and the faculty members for their cooperation.

Finally, we thank our families and friends for their patience and moral support during the course of this project. The open-source community deserves recognition as well — this project is built almost entirely on open-source libraries and openly available pretrained machine learning models, without which the work would not have been possible within the scope of a minor project.

ABSTRACT

The volume of long-form digital documents — research papers, project specifications, legal contracts, government reports and scanned records — grows faster than any person can read them. Automatic text summarization addresses this by condensing a document into a short, readable form that preserves its essential information.

This project presents **AI Document Summarizer**, a full-stack web application that ingests documents in **PDF, DOCX and TXT** formats and produces summaries using **three complementary strategies**: *extractive*, *abstractive* and *hybrid*. The system extracts text from the uploaded file, falls back to **Optical Character Recognition (Tesseract)** for scanned PDF pages that carry no text layer, cleans the raw text into flowing prose, and then applies the user-selected summarization strategy. Summaries can optionally be scored against a human-written reference using **ROUGE-1, ROUGE-2, ROUGE-L and BLEU** metrics.

The **extractive** strategy embeds every sentence using the `all-MiniLM-L6-v2` sentence-transformer model, builds a sentence-similarity graph and runs **TextRank** (PageRank) over it to select the most representative sentences verbatim — an approach that cannot hallucinate because no sentence is ever rewritten. The **abstractive** strategy uses the `facebook/bart-large-cnn` sequence-to-sequence transformer to generate new, fluent sentences, applying a **map-reduce** scheme with sentence-aligned overlapping chunks so that documents exceeding the model's 1024-token limit can still be summarized. The **hybrid** strategy first filters the document to its highest-ranked 50% of sentences using TextRank and then feeds only those sentences to BART, constraining generation to factually central content and reducing the risk of hallucination.

The system is implemented as a **FastAPI** backend exposing four REST endpoints and a **React** single-page frontend built with Vite and Tailwind CSS. Machine learning models are loaded lazily as process-level singletons and blocking inference is dispatched to worker threads so the HTTP event loop remains responsive. The backend is covered by a test suite of **89 automated tests** spanning extraction, cleaning, sentence segmentation, chunking, OCR orchestration and the API layer.

Keywords: text summarization, extractive summarization, abstractive summarization, TextRank, BART, OCR, ROUGE, BLEU, FastAPI, React.

TABLE OF CONTENTS

Chapter	Title	Page
	Certificate	ii
	Declaration	iii
	Acknowledgement	iv
	Abstract	v
	List of Figures	viii
	List of Tables	viii
1	**Introduction**	1
1.1	Overview	1
1.2	Motivation and Problem Statement	1
1.3	Objectives	2
1.4	Scope of the Project	2
1.5	Organisation of the Report	3
2	**Literature Review and Background**	4
2.1	Text Summarization: A Historical View	4
2.2	Extractive Approaches	4
2.3	Abstractive Approaches and Transformer Models	5
2.4	Hybrid Approaches	6
2.5	Evaluation Metrics	6
2.6	Optical Character Recognition	7
3	**System Analysis**	8
3.1	Existing Systems and Their Limitations	8
3.2	Proposed System	8
3.3	Feasibility Study	9
3.4	Hardware and Software Requirements	10
4	**System Design**	11
4.1	Architecture Overview	11
4.2	Module Decomposition	12
4.3	Data Flow	13
4.4	Database Design (Server-Side Document Store)	14

Chapter	Title	Page
4.5	API Design	14
4.6	Key Design Decisions	15
5	**Implementation**	17
5.1	Technology Stack	17
5.2	Document Ingestion and OCR	17
5.3	Text Cleaning	19
5.4	Sentence Segmentation and Chunking	20
5.5	Extractive Summarization	21
5.6	Abstractive Summarization	22
5.7	Hybrid Summarization	23
5.8	Evaluation Module	24
5.9	Frontend Implementation	25
6	**Testing**	27
6.1	Testing Strategy	27
6.2	Test Suite Composition	27
6.3	Techniques for Portability	28
6.4	Sample Test Cases	29
7	**Results and Discussion**	30
7.1	Functional Results	30
7.2	Behaviour of the Three Strategies	30
7.3	Performance Observations	31
7.4	Limitations	31
8	**Conclusion and Future Scope**	33
8.1	Conclusion	33
8.2	Future Scope	33
	References	34
	Appendix A — API Endpoint Reference	35
	Appendix B — Setup and Execution	36

LIST OF FIGURES

Figure	Title
4.1	High-level system architecture
4.2	End-to-end processing pipeline (5 stages)
4.3	Module dependency graph
5.1	Extractive summarization flow (embeddings → TextRank)
5.2	Abstractive map-reduce flow
5.3	Hybrid summarization flow
5.4	Frontend page state machine

LIST OF TABLES

Table	Title
3.1	Hardware requirements
3.2	Software requirements
4.1	API endpoint summary
4.2	Summarization modes and their trade-offs
5.1	Cleaning operations and their effects
5.2	Summary length parameters
6.1	Test suite composition by module
6.2	Representative test cases
7.1	ROUGE and BLEU interpretation ranges

CHAPTER 1 — INTRODUCTION

1.1 Overview

Reading is no longer the bottleneck it once was for access to information — but it is now the bottleneck for *processing* information. A single research paper runs to ten or more pages, a project specification document to fifty, and a government or legal report to hundreds. A reader who needs to know whether a document is relevant, or what its central claims are, cannot afford to read every word.

AI Document Summarizer is a web application that accepts such documents and returns a short, information-dense summary. It supports the three file formats most commonly encountered in academic and professional work — **PDF**, **DOCX** and **TXT** — and offers three summarization strategies with different trade-offs between factual fidelity and linguistic fluency.

The application is deliberately structured as a pipeline rather than a single model call. Every document passes through the same five stages: **ingestion, cleaning, chunking, summarization and evaluation**. The summarization stage is the only one that differs by user choice; the rest of the pipeline is shared. This structure is what allows a scanned, badly formatted PDF and a clean TXT file to be handled by the same downstream code.

1.2 Motivation and Problem Statement

Manual summarization is slow, subjective and expensive. A professional summarising a 40-page document may spend an hour on it; the same document summarised by a colleague will produce a different result. Machine summarization is fast, consistent and cheap, and — while it will never replace careful human reading — it is well suited to triage, indexing and first-pass comprehension.

The problem this project addresses can be stated precisely:

> **Given an arbitrary long-form document in PDF, DOCX or TXT format — possibly scanned, possibly poorly formatted — produce a concise summary of it, while allowing the user to choose between guaranteed factual fidelity and fluent paraphrasing, and optionally measuring the result against a human-written reference.**

Three sub-problems follow from this statement and shaped the design of the system:

1. **Extraction.** Documents are not always stored as clean text. PDFs are page layout grids, scanned PDFs contain no text at all, and DOCX files split content across paragraphs and tables. A single summarizer cannot begin until these differences are normalised away.
2. **Model limits.** The abstractive model used — BART — reads at most 1024 tokens. A twenty-page document is far longer. The document must therefore be split, summarized piecewise and the results recombined, without cutting sentences in half in the process.
3. **Hallucination.** Abstractive models generate new sentences, and generated sentences can assert things the source document never said. For academic, legal or medical documents this is a serious

failure mode. The system must offer a strategy that cannot hallucinate, and a middle path that constrains generation to factually central content.

1.3 Objectives

The objectives of the project are:

1. To build a document ingestion subsystem that extracts text from **PDF, DOCX and TXT** files, including an **OCR fallback** for scanned PDF pages that carry no text layer.
2. To implement a **text cleaning** stage that converts extracted page layout back into flowing prose by removing page numbers and repeated headers/footers, re-joining words broken across line breaks, and normalising characters.
3. To implement **sentence-aware, overlapping chunking** so that documents longer than the model's context window can be processed without fragmenting sentences.
4. To implement **extractive summarization** using sentence embeddings and the **TextRank** graph-ranking algorithm, with a guarantee that every output sentence is quoted verbatim from the source.
5. To implement **abstractive summarization** using the **BART** sequence-to-sequence transformer with a **map-reduce** scheme for long documents.
6. To implement **hybrid summarization** that pre-filters with TextRank before abstractive condensation, reducing hallucination risk while retaining fluency.
7. To implement **automatic evaluation** of a generated summary against a human reference using **ROUGE-1, ROUGE-2, ROUGE-L and BLEU**.
8. To build a responsive **single-page web interface** for upload, strategy selection, summarization and evaluation.
9. To validate the system with an **automated test suite** covering extraction, cleaning, segmentation, chunking, OCR orchestration and the HTTP API.

1.4 Scope of the Project

Included in scope:

- Document formats: PDF (digital and scanned), DOCX (paragraphs *and* tables), TXT (with encoding detection).
- Three summarization strategies: extractive, abstractive and hybrid.
- Three summary length settings: short, medium and long.
- OCR for English-language scanned pages, using Tesseract.
- Reference-based evaluation with ROUGE and BLEU.
- A local development deployment: FastAPI backend and React frontend running on the same machine.

Excluded from scope:

- Languages other than English (both OCR and the pretrained models are configured for English).
- Multi-document or query-focused summarization.

- User accounts, authentication or a persistent summary history.
- Containerised or cloud deployment (identified as a later phase; see §8.2).
- GPU inference (the system runs on CPU with configurable device support).

1.5 Organisation of the Report

Chapter 2 reviews the literature behind the project: the extractive/abstractive distinction, TextRank, transformer models, the origin and purpose of the ROUGE and BLEU metrics, and OCR. **Chapter 3** analyses the problem, compares existing tools and records the requirements. **Chapter 4** presents the architecture, module decomposition, data flow and API design. **Chapter 5** documents the implementation of each module with the algorithms and parameters actually used. **Chapter 6** describes the testing strategy and the composition of the test suite. **Chapter 7** discusses results and limitations. **Chapter 8** concludes and outlines future work. Two appendices give the API reference and the setup instructions.

CHAPTER 2 — LITERATURE REVIEW AND BACKGROUND

2.1 Text Summarization: A Historical View

Automatic text summarization is one of the oldest problems in natural language processing. Luhn's work in 1958 proposed scoring sentences by the frequency of significant words they contain and selecting the highest-scoring sentences as a summary — an approach that is *extractive* in nature and recognisably an ancestor of the methods used today. The field has since divided into two broad families, with a third hybrid family combining them.

A further distinction, introduced by Mani and Maybury and now standard, separates **generic** summarization (summarise the document as a whole) from **query-focused** summarization (summarise with respect to a question). This project is generic.

2.2 Extractive Approaches

Extractive summarization selects a subset of the source document's sentences and returns them unchanged. Its central virtue is *fidelity*: because no text is generated, the summary cannot contain a fact that was not in the source. Its central weakness is *cohesion*: a summary assembled from sentences scattered across a document may read awkwardly and may repeat itself.

Early extractive methods scored sentences by lexical signals — word frequency (Luhn), position in the document, presence of cue phrases such as "in conclusion", and title-word overlap (Edmundson's four-feature method). These methods are fast and need no training data, but they score sentences individually and therefore cannot capture the fact that a sentence is important because *many other sentences in the document are about the same thing*.

2.2.1 TextRank

TextRank, introduced by Mihalcea and Tarau in 2004, addresses this limitation by treating summarization as a graph-ranking problem. The algorithm:

1. builds a graph in which each **sentence is a node**;
2. draws an **edge between two sentences** whose content overlaps;
3. weights each edge by the **similarity** of the two sentences;
4. runs the **PageRank** algorithm over the graph, and
5. selects the highest-ranked sentences for the summary.

TextRank's PageRank step is the same algorithm Brin and Page introduced for web search, applied to sentences instead of pages. It is **unsupervised**: no training data and no domain adaptation are required. Crucially, the score it assigns reflects *centrality*: a sentence scores highly if it is similar to many other similar sentences, which in practice identifies sentences that express the document's recurrent themes.

The original TextRank computes similarity using lexical overlap between sentences, counting shared words. This project replaces that similarity function with **dense sentence embeddings** (§2.2.2), which

measure *meaning* rather than surface word overlap: two sentences that say the same thing in different words will have a high similarity under embeddings but possibly zero under lexical overlap.

2.2.2 Sentence Embeddings

A sentence embedding is a fixed-length numeric vector that represents a sentence's meaning, such that sentences with similar meanings have vectors that point in similar directions. Modern sentence embeddings are produced by transformer encoder models trained specifically so that semantically related sentences are close together in vector space.

This project uses `all-MiniLM-L6-v2`, a small sentence-transformer model. It produces **384-dimensional** embeddings and its weights occupy roughly 90 MB. It was chosen over larger alternatives because the extractive stage must embed *every sentence* in the document — a single embedding call per sentence — and a small, fast encoder keeps that cost acceptable on CPU without a material loss of quality for this task.

Embeddings are compared using **cosine similarity**, which measures the angle between two vectors and is therefore insensitive to their magnitude:

```
cosine(u, v) = (u · v) / (‖u‖ ‖v‖)
```

If the vectors are first **normalised to unit length**, the dot product equals the cosine similarity, which is why the implementation normalises embeddings at encode time and then computes similarities with a single matrix multiplication.

2.3 Abstractive Approaches and Transformer Models

Abstractive summarization generates *new* sentences that convey the document's content. It can compress more aggressively than the extractive approach and produces more natural prose, because it is not constrained to reuse the source's phrasing. The price is that the output is generated text and may therefore contain **hallucinations** — fluent assertions unsupported by the source.

2.3.1 Sequence-to-Sequence Models

Abstractive summarization became practical with the **sequence-to-sequence (seq2seq)** architecture. A seq2seq model has an **encoder** that reads the input text into an internal representation and a **decoder** that generates the output text one token at a time from that representation. For summarization, the input is the document and the output is the summary.

2.3.2 The Transformer and BART

The **Transformer** architecture (Vaswani et al., 2017) replaced the recurrent networks used in early seq2seq models with **self-attention**, allowing the model to relate any two positions in the input directly rather than through a sequence of recurrent steps. This made it possible to train much larger models on much more text.

BART (Lewis et al., 2019) is a denoising seq2seq model: it is pretrained by corrupting text and learning to reconstruct the original, which forces the encoder to build a useful representation of language and the decoder to generate fluent text from it. `facebook/bart-large-cnn` is a BART model fine-tuned specifically

on the CNN/DailyMail news summarization dataset, where it established strong performance on abstractive summarization. Its input limit is **1024 tokens**, and in practice the implementation caps single-pass input at **900 tokens** to leave margin.

2.3.3 Map-Reduce for Long Documents

Because a single BART pass cannot cover a long document, the system applies the **map-reduce** pattern, borrowed from distributed data processing:

- **Map** — split the document into chunks small enough for one model pass and summarize each chunk independently, producing a partial summary per chunk.
- **Reduce** — concatenate the partial summaries into one text and, if that text is still longer than the model can accept, condense it once more.

This is exactly the strategy described in the summarization literature for handling documents that exceed a model's context window, and it is the strategy implemented in §5.6. The name map-reduce is apt but the implementation is sequential: reducing a very long document means summarizing each chunk, joining the results, and — if necessary — summarizing the joined result once more.

2.4 Hybrid Approaches

Hybrid summarization combines extractive selection with abstractive generation, on the reasoning that each method's strength can compensate for the other's weakness. The typical formulation is **extract-then-abstract**: use an extractive scorer to identify the document's most important sentences, discard the rest, and pass only the important sentences to the abstractive model.

The benefit is twofold. First, **compression before generation**: BART receives a shorter, denser input and therefore has less opportunity to drift onto unimportant content. Second, **constrained generation**: sentences excluded by the extractive filter cannot be paraphrased into the summary, which materially reduces the space of possible hallucinations. The trade-off is that a sentence the extractive scorer ranked low is permanently excluded — if the ranker is wrong, the abstractive model cannot recover the missed content.

This project implements the extract-then-abstract formulation, retaining the **top 50%** of sentences by TextRank score before the abstractive pass (§5.7).

2.5 Evaluation Metrics

Summarization quality is difficult to measure automatically, because there is no single correct summary of a document. The field's practical solution is **reference-based evaluation**: a human-written summary of the same document is treated as ground truth, and the system's summary is scored by how much of that reference it recovers. Two metric families are used in this project.

2.5.1 ROUGE

ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*), introduced by Lin in 2004, measures the overlap between a generated summary and a reference summary. The variants relevant here are:

- **ROUGE-1** — overlap of **unigrams** (single words). A high score indicates that the summary covers the reference's vocabulary and topics.
- **ROUGE-2** — overlap of **bigrams** (word pairs). This is a stricter test of local phrasing: two texts can share most of their words while sharing far fewer word pairs.
- **ROUGE-L** — based on the **Longest Common Subsequence**, the longest sequence of words that appears in both texts in the same order but not necessarily contiguously. This rewards the preservation of word order and sentence structure without demanding exact n-gram matches.

For each variant the system computes precision *P*, recall *R* and **F1** — the harmonic mean of the two — and reports the **F1** score, a number between 0.0 and 1.0.

```
F1 = 2 · (P · R) / (P + R)
```

2.5.2 BLEU

BLEU (*Bilingual Evaluation Understudy*), introduced by Papineni et al. in 2002 for machine translation evaluation, is a **precision-oriented** metric. It computes the proportion of n-grams in the generated text (for n = 1 to 4) that also appear in the reference, takes the geometric mean of those precisions, and multiplies by a **brevity penalty** that discourages suspiciously short output:

```
BLEU = BP · exp( Σ wn · log pn )
```

The score is reported on a 0–100 scale. The brevity penalty is the reason a very short summary cannot score highly by being trivially precise.

2.5.3 Why Both

ROUGE is **recall-oriented** — it answers "how much of the reference did the summary capture?" BLEU is **precision-oriented** — it answers "how much of the summary is correct?" A summary that copies the reference scores well on both; a very short summary may score well on BLEU's per-gram precision but is penalised by the brevity penalty and scores poorly on ROUGE recall. Using both gives a fuller picture than either alone. Both are implemented using the standard `rouge-score` and `sacrebleu` packages (§5.8) rather than reimplemented, so results are comparable with published work.

2.6 Optical Character Recognition

Optical Character Recognition (OCR) converts the *image* of text into machine-readable characters. It is required whenever a document is a scan: a PDF produced by a scanner typically stores one photograph per page, so there is no text layer for a PDF library to read.

The **Tesseract** OCR engine, originally developed at HP and now maintained by Google, is the standard open-source choice. It segments a page image into lines and words, classifies the characters, and applies a language model to correct implausible sequences. Modern versions use an LSTM-based recogniser. Two practical points shaped this project's use of it:

- **Resolution matters.** Tesseract's default assumption of 72 DPI is too low for small text; rendering at **200 DPI** before recognition markedly improves accuracy. This is why the system renders pages at `OCR_DPI = 200`.
- **It is an external program, not a library.** The Python package `pytesseract` is only a wrapper; the `tesseract` binary must be installed separately. The system therefore treats OCR as **optional** and degrades gracefully when it is missing (§5.2), rather than failing.

CHAPTER 3 — SYSTEM ANALYSIS

3.1 Existing Systems and Their Limitations

Several categories of existing tool address parts of this problem. Their limitations define the gap this project fills.

Category	Examples	Strengths	Limitations
Online summarizer services	Browser-based summarization websites	No installation; instant	Require uploading confidential documents to a third party; no control over the algorithm; usually a single summarization method
Abstractive-only model interfaces	Direct use of a summarization model	High fluency	Cannot summarise documents longer than the model's context window; no OCR; hallucination risk unaddressed
Extractive-only libraries	Classical TextRank implementations	Fast; factually safe	Lexical similarity misses paraphrases; output can read awkwardly; no evaluation built in
PDF reader "summarize" features	Features bundled into document readers	Convenient	Tied to one format; no scanned-document support; closed algorithm
General-purpose language model chat	Conversational assistants	Flexible	Variable, non-reproducible output; summarising a long document requires manual chunking; no ROUGE/BLEU measurement; cost per document

Three specific gaps emerge:

1. **No single tool combines** robust multi-format ingestion (including OCR), multiple summarization strategies, and built-in reference-based evaluation.
2. **Document length is routinely underestimated.** Tools that call a summarization model directly silently truncate long documents, discarding everything past the model's context window.
3. **Hallucination is rarely surfaced to the user.** Tools that generate text typically do not offer a strategy that *cannot* invent facts, nor do they explain the trade-off.

3.2 Proposed System

The proposed system is a self-hosted web application with a clean separation between an HTTP API and a set of pure algorithmic services. Its distinguishing features are:

- **Multi-format ingestion with OCR fallback.** PDF, DOCX and TXT are accepted. A PDF page whose text layer is empty is automatically routed to Tesseract; a page that already has text never pays the OCR cost. Partly-scanned documents keep their digital pages and have only the image pages filled in. Each extraction reports which route it took.
- **Three summarization strategies** with an explicit trade-off between fidelity and fluency, so the user chooses the risk they are willing to accept rather than inheriting one.

- **Correct handling of long documents.** Sentence-aware overlapping chunking plus map-reduce means a document far longer than the model's context window is summarized in full, not truncated.
- **Built-in reference-based evaluation.** ROUGE-1/2/L and BLEU are computed on demand against a user-supplied reference summary.
- **Server-side document store.** The document is extracted and cleaned once; subsequent summarize or evaluate calls reference it by id, so changing strategy or length never re-uploads the file.
- **Graceful degradation.** Both external dependencies — the Tesseract binary and the NLTK sentence tokenizer data — are optional. Their absence disables a feature (scanned-PDF OCR, or higher-quality sentence splitting) but never crashes the application.

3.2.1 Comparison of the Three Strategies

Table 4.2 — Summarization modes and their trade-offs

Strategy	Mechanism	Factual Fidelity	Fluency	Relative Speed
Extractive	Sentence embeddings → cosine similarity → TextRank → top-*k* sentences, returned verbatim	**Highest** — cannot hallucinate; every sentence is quoted from the source	Lowest — output may read disjointedly, since sentences are lifted out of context	Fastest — one embedding pass plus a PageRank computation; no text generation
Abstractive	BART seq2seq generation, with map-reduce over chunks for long documents	Lowest — generated text can assert unsupported facts	**Highest** — produces new, natural sentences that synthesise concepts	Slowest — autoregressive generation per chunk, multiplied by the number of chunks
Hybrid	TextRank keeps the top 50% of sentences → BART condenses them	Compromise — generation is constrained to factually central sentences, shrinking the space of possible hallucinations	High — retains abstractive fluency while staying close to the selected content	Intermediate — an embedding/ranking pass followed by generation over a *shorter* input

3.3 Feasibility Study

3.3.1 Technical Feasibility

The project depends on mature, widely-used open-source components: FastAPI for the API layer, React for the interface, PyMuPDF and python-docx for extraction, Tesseract for OCR, the `transformers` and `sentence-transformers` libraries for models, and `rouge-score` and `sacrebleu` for evaluation. All are actively maintained and freely available. No component required original machine learning research. Two constraints were identified and are handled explicitly: model weights must be downloaded once (~90 MB and ~1.6 GB) and cached, and CPU inference on long documents is slow, which the interface communicates through generous timeouts and progress states.

3.3.2 Economic Feasibility

The entire system runs on commodity hardware with no licences and no cloud services. There is no per-document cost, which contrasts favourably with API-based summarization. The only recurring cost is electricity during inference.

3.3.3 Operational Feasibility

The application runs as two local processes (a Python API server and a Node development server) and is operated entirely through a browser. No specialised knowledge is required of the end user, and no user management or database administration is involved.

3.3.4 Time Feasibility

The project was executed in explicitly ordered phases: setup, ingestion, OCR, preprocessing, extractive summarization, abstractive summarization, hybrid summarization, evaluation, frontend, testing, and documentation. Each phase produced a working increment, so the system was demonstrable at every stage rather than only at the end.

3.4 Hardware and Software Requirements

3.4.1 Hardware Requirements

Table 3.1 — Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core x86-64, 2 GHz	Quad-core or better, 2.5 GHz+
RAM	8 GB	16 GB (the BART model occupies roughly 1.6 GB of weights plus runtime overhead)
Disk space	4 GB free	8 GB free (model cache is the dominant consumer)
Display	1280 × 720	1920 × 1080
Network	Required for the one-time model download	—

3.4.2 Software Requirements

Table 3.2 — Software requirements

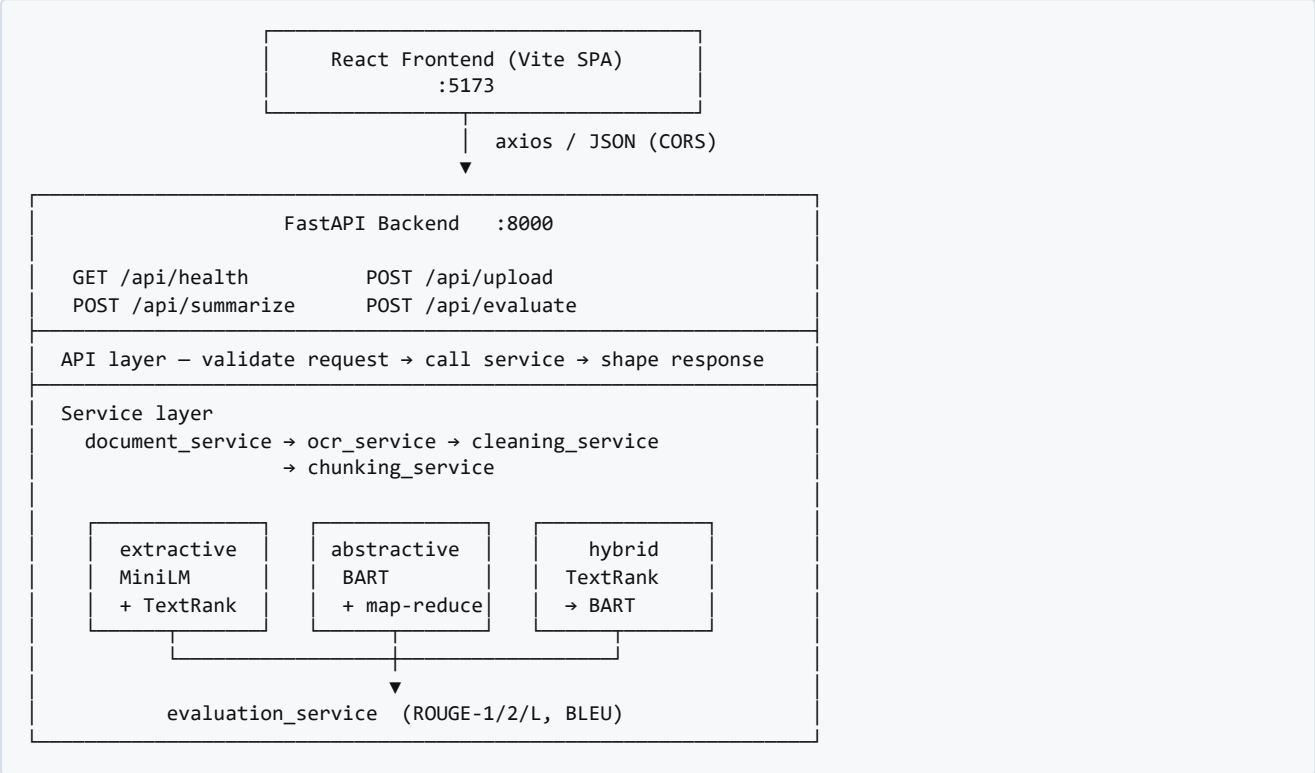
Layer	Requirement
Operating system	Windows 10/11, Linux or macOS
Language runtime (backend)	Python 3.11 (recommended for machine-learning wheel availability)
Language runtime (frontend)	Node.js 20.19+ or 22.12+
External program	Tesseract OCR 5.x — required only for scanned PDFs; installed at `C:\Program Files\Tesseract-OCR` on Windows
Backend framework	FastAPI, Uvicorn, Pydantic v2
Frontend framework	React 19, Vite 8, Tailwind CSS v4, axios
Extraction libraries	PyMuPDF (`fitz`), python-docx
Machine learning	PyTorch, `transformers` , `sentence-transformers` , NetworkX, NLTK
Evaluation libraries	`rouge-score` , `sacrebleu`
Testing	pytest, httpx
Browser	Any modern browser (Chrome, Edge, Firefox)

CHAPTER 4 — SYSTEM DESIGN

4.1 Architecture Overview

The system is a **two-tier client–server application**. The React single-page application runs in the browser and communicates with a FastAPI backend over HTTP using JSON. Because the two run on different ports they are different origins, so the backend configures **CORS** (Cross-Origin Resource Sharing) to permit the frontend's requests.

Figure 4.1 — High-level system architecture



4.2 Module Decomposition

The backend is split into four packages with a strict rule: **routers hold no algorithms**.

Package	Responsibility
`app/api/`	Four HTTP routers — `health`, `upload`, `summarize`, `evaluate`. Each validates its request against a Pydantic model, delegates to a service function, and shapes the response.
`app/services/`	All business logic, as plain Python functions with no HTTP dependency. Documents, OCR, cleaning, chunking, the three summarizers and evaluation.
`app/models/schemas.py`	The Pydantic request/response contracts that define the API's public shape.
`app/utils/`	Small stateless helpers: filename safety and size limits (`file_utils`), text counting and previews (`text_utils`).
`app/config.py`	All tunable settings, loaded from environment variables or a `.env` file, with derived values (resolved paths, parsed lists) exposed as properties.
`app/main.py`	The application factory: logging, lifespan, CORS, the two exception handlers, and router registration.

Figure 4.3 — Module dependency graph

```
main.py
  └── api/{health, upload, summarize, evaluate}
        ├── health      ──► services/ocr_service
        ├── upload      ──► services/document_service ──► ocr_service
        │                                             ──► cleaning_service
        │                                             ──► chunking_service
        ├── summarize   ──► services/{extractive, abstractive, hybrid}_service
        │                                             ──► document_service
        └── evaluate    ──► evaluation_service, document_service
```

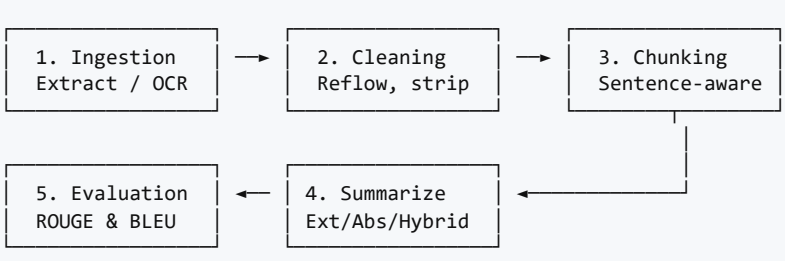
The separation is what makes the algorithms testable: every summarizer is a function that takes a string and returns a string, and can be exercised without starting an HTTP server. The test suite described in Chapter 6 relies on exactly this.

4.2.1 Services in Detail

Service	Function	Description
`document_service`	`extract_text`, `extract_and_clean`, `save_document`, `load_document`	Extracts text by file type, dispatches scanned PDF pages to OCR, cleans the result, and stores the cleaned text and its metadata under a generated `document_id`.
`ocr_service`	`availability`, `render_page`, `ocr_page`	Renders a PDF page at the configured DPI and recognises its text with Tesseract. Reports availability explicitly so callers can explain a missing installation rather than raising an opaque error.
`cleaning_service`	`clean_text`	Normalises characters, removes page numbers and repeated headers/footers, re-joins hyphenated line breaks and reflows wrapped lines into paragraphs.
`chunking_service`	`split_sentences`, `chunk_text`	Sentence segmentation (NLTK Punkt with a regex fallback) and sentence-aligned overlapping chunking.
`extractive_service`	`rank_sentences`, `summarize`	Embeds sentences with MiniLM, builds the similarity graph and runs PageRank, then selects the top-ranked sentences in their original order.
`abstractive_service`	`summarize`	BART generation, with map-reduce over chunks when the input exceeds the single-pass token cap.
`hybrid_service`	`summarize`	TextRank filtering to the top 50% of sentences followed by abstractive condensation.
`evaluation_service`	`evaluate`	Computes ROUGE-1/2/L and BLEU between a generated summary and a reference.

4.3 Data Flow

Figure 4.2 — End-to-end processing pipeline (five stages)



The flow for a single document is as follows:

1. **Upload** — the browser posts the file to `POST /api/upload`. The filename is sanitised, the extension checked, and the bytes written to a temporary file under the size limit (20 MB).
2. **Extraction** — `document_service` dispatches on file type. For a PDF, each page's text layer is read; any page below the character threshold is treated as an image and routed to OCR. For a DOCX, paragraphs

and tables are read. For a TXT, the file is decoded with encoding detection.

3. **Cleaning** — the extracted text is normalised, de-paginated and reflowed into prose.
4. **Storage** — the cleaned text is written to `uploads/<id>.txt` and its metadata to `uploads/<id>.json`; the temporary upload is deleted. The response returns the `document_id` and metadata — **never the text**.
5. **Summarization** (on request) — the browser posts the `document_id`, a mode and a length. The service loads the text, dispatches to the selected summarizer in a worker thread, and returns the summary.
6. **Evaluation** (optional) — the browser posts the `document_id` and a reference summary. The document is summarized and the result scored against the reference.

Passing only the `document_id` at steps 5 and 6 is a deliberate design choice. The client never re-uploads a 20 MB PDF simply because the user switched from extractive to abstractive mode, and the document text never crosses the network twice.

4.4 Database Design (Server-Side Document Store)

The system uses no database management system. Because the project has no user accounts and no requirement to retain history, a **file-based document store** is sufficient and avoids an unnecessary dependency. The store is a directory (`uploads/`) containing, per document, a pair of files named after a generated identifier:

File	Content
`uploads/<document_id>.txt`	The cleaned document text, UTF-8 encoded.
`uploads/<document_id>.json`	Metadata: filename, file type, character and word counts, chunk count, page count, whether OCR was used, extraction method, and cleaning statistics.

The `document_id` is a **UUID4 hex string** — 128 bits of randomness, which makes identifiers unguessable and collisions negligible. It serves simultaneously as the on-disk filename stem and as the API handle. Because the original uploaded binary is deleted immediately after extraction, the store contains text only.

A consequence worth stating honestly: the store is **process-local and not persistent across a cleanup of the uploads directory**. If the directory is cleared, a subsequent `/api/summarize` for that id returns HTTP 404 with the message "Document not found or no longer available. Please upload it again." This is acceptable for a local single-user tool; a persistent store is identified in future scope (§8.2).

4.5 API Design

Table 4.1 — API endpoint summary

Method	Path	Purpose
`GET`	`/api/health`	Liveness probe. Reports service name, version, Python version, platform, timestamp, and OCR availability. Used by the frontend to confirm the backend is reachable before the user attempts an upload.
`POST`	`/api/upload`	Accepts a multipart file upload. Validates and extracts, applies OCR where needed, cleans, stores, and returns metadata only.
`POST`	`/api/summarize`	Accepts a `document_id`, a `mode` and a `length`; returns the generated summary.
`POST`	`/api/evaluate`	Accepts a `document_id`, a `reference` summary, a `mode` and a `length`; returns the generated summary together with ROUGE and BLEU scores.

4.5.1 Contract-First with Pydantic

Every request and response is declared as a Pydantic model in `app/models/schemas.py`. This yields three benefits simultaneously: FastAPI validates incoming requests against the model, serialises outgoing responses *through* the model (so only declared fields are ever sent), and generates the OpenAPI documentation at `/docs` from the same definitions.

Two fields are declared as **typed enumerations** rather than plain strings:

```
SummarizationMode = Literal["extractive", "abstractive", "hybrid"]
SummaryLength     = Literal["short", "medium", "long"]
```

Declaring these once means a typo such as `mode="extractiv"` is rejected at the boundary with **HTTP 422** rather than failing deep inside a service with an obscure error.

4.5.2 Uniform Error Responses

Every failing endpoint returns the same JSON shape — `{ "detail": "<message>" }` — where `detail` is a sentence written for the end user, not a stack trace. Two application-wide handlers enforce this:

- The **validation handler** collapses FastAPI's verbose 422 body (a list of error objects) into a single readable string, because the frontend only ever has to render one message.
- The **catch-all handler** logs the full traceback server-side and returns a generic 500 to the client, so internal details are never leaked. Debugging is unaffected because the log retains everything.

4.6 Key Design Decisions

1. `api/` and `services/` are separated. Routers contain no algorithms. This is not architectural decoration — it is what allows every summarizer, the cleaner and the chunker to be unit-tested as plain functions, without an HTTP client.

2. The document store is server-side. The upload endpoint extracts once and returns only an id plus metadata (§4.4). Switching mode or length costs one small JSON request, not a re-upload.

3. Models are process-level singletons. Both models are loaded through a cached getter (`@lru_cache(maxsize=1)`) so each is loaded once per process, not once per request. Reloading BART per

request would add roughly ten seconds to every call. The heavy imports (`torch`, `transformers`, `sentence_transformers`) happen *inside* the cached getter, so importing a service module does not pull PyTorch into memory; only the first summarization request pays that cost.

4. Blocking work runs in a thread pool. Model inference and graph algorithms are synchronous, CPU-bound operations that would otherwise block FastAPI's event loop and stall concurrent requests. Routers therefore dispatch them with `await run_in_threadpool(...)` — this is why `/api/health` continues to answer while a long document is being summarized.

5. Chunking exists because of a hard model limit. BART reads 1024 tokens. The design response is not to truncate the document but to split it on sentence boundaries, summarize the pieces and combine the results (§5.6).

6. Optional dependencies degrade rather than crash. Tesseract and the NLTK punkt data are both external to `pip install`. The system probes for each and adapts: without Tesseract, digital documents work normally and a scanned upload is rejected with a message naming the fix (`ocr_service.availability()` exists precisely so this can be reported rather than thrown); without punkt, a regex sentence splitter handles abbreviations, initials, decimals and dotted forms.

7. Cleaning is deliberately conservative. Removing a real line of content is a far worse failure than leaving a stray footer behind. The cleaner therefore removes a line only when it is structurally meaningless (a page number) or provably repeated (a running header) — and a repeated *full sentence* is never treated as a header, however often it occurs.

CHAPTER 5 — IMPLEMENTATION

5.1 Technology Stack

Layer	Technology	Role in the project
Frontend	React 19, Vite 8, Tailwind CSS v4, axios	Single-page interface with state-driven rendering; axios for the API calls, with all error normalisation in one module
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic v2	Async HTTP API with automatic validation, OpenAPI documentation and typed contracts
Extraction	PyMuPDF (`fitz`), python-docx	PDF text layer and page rendering; DOCX paragraphs and tables
OCR	Tesseract via `pytesseract`, Pillow	Recognition of scanned PDF pages
Embeddings	`sentence-transformers` (`all-MiniLM-L6-v2`)	384-dimensional sentence embeddings for extractive ranking
Abstractive model	`transformers` (`facebook/bart-large-cnn`)	Sequence-to-sequence summarization
Graph	NetworkX	PageRank implementation used for TextRank
Numerics	NumPy, scikit-learn	Similarity matrix computation
Segmentation	NLTK (Punkt)	Sentence segmentation, with a regex fallback
Evaluation	`rouge-score`, `sacrebleu`	ROUGE-1/2/L and BLEU
Testing	pytest, httpx	Automated test suite

5.2 Document Ingestion and OCR

The ingestion module, `document_service`, is responsible for turning an arbitrary input file into clean prose. Its entry point dispatches on file type:

```
def extract_text(path: Path, file_type: str) -> ExtractionResult:
    if file_type == "pdf":
        return _extract_pdf(path)
    if file_type == "docx":
        return _extract_docx(path)
    if file_type == "txt":
        return _extract_txt(path)
    raise HTTPException(status_code=400, detail=f"Cannot process '{file_type}' files.")
```

The upload endpoint uses `extract_and_clean`, which runs extraction and then cleaning. Extraction remains a separate public function because it is meaningful on its own and is tested as such.

5.2.1 PDF Extraction with OCR Fallback

The central logic of `_extract_pdf` is the decision of which pages need OCR:

```
page_texts = [page.get_text("text") or "" for page in document]

scanned_pages = [
    number
    for number, page_text in enumerate(page_texts, start=1)
    if len(page_text.strip()) < settings.OCR_MIN_CHARS_PER_PAGE
]
```

A page whose text layer yields fewer than `OCR_MIN_CHARS_PER_PAGE` (default **80**) characters is treated as an image. This threshold is what decides whether OCR runs at all, and its effect is that **a normal digital PDF costs nothing extra** — its pages exceed the threshold and are never re-rendered. Partial scans are handled naturally: the digital pages keep their text and only the image pages have their (empty) text-layer entries overwritten with the OCR result.

Two implementation constraints are handled explicitly. First, OCR must run **while the document is open**, because it renders pages and PyMuPDF cannot render a closed document; the code therefore performs OCR inside the `try` block and closes the document in a `finally`. Second, closing the native handle is necessary on Windows or the temporary file cannot be deleted.

OCR failure is deliberately **non-fatal per page**:

```
for index, number in enumerate(scanned_pages):
    try:
        page_text = ocr_service.ocr_page(document[number - 1])
    except ocr_service.OcrUnavailableError as exc:
        # Tesseract gave out part-way through — stop rather than retrying every page.
        logger.warning("OCR stopped at page %d: %s", number, exc)
        failed.extend(scanned_pages[index:])
        return OcrOutcome(True, recovered, failed, unavailable_reason=str(exc))
    except Exception as exc:
        logger.warning("OCR failed on page %d: %s", number, exc)
        failed.append(number)
        continue
```

A mostly-digital PDF with one unreadable scan still uploads. Only a *completely* unavailable Tesseract is reported upward, and even then the caller decides whether that is an error. Failures are surfaced to the user as warnings rather than exceptions, so the user knows which pages are missing from the summary.

The extraction method string is keyed on whether the document had a real text layer *anywhere*, rather than on how many pages OCR recovered:

```
def _pdf_method(text_layer_pages: int, outcome: OcrOutcome) -> str:
    if not outcome.recovered:
        return "PyMuPDF text layer"
    if text_layer_pages == 0:
        return f"Tesseract OCR ({len(outcome.recovered)} page(s))"
    return f"PyMuPDF text layer + Tesseract OCR ({len(outcome.recovered)} page(s))"
```

This matters for honesty: a fully scanned PDF whose OCR partly failed is still an OCR extraction, and labelling it "PyMuPDF text layer" would misreport how the text was obtained.

5.2.2 DOCX and TXT Extraction

DOCX extraction reads paragraphs **and tables**, because tables frequently hold the substance of a report — results, metrics, comparisons — and a summary built only from paragraphs would miss them:

```
blocks = [p.text.strip() for p in document.paragraphs if p.text.strip()]

for table in document.tables:
    for row in table.rows:
        cells = [cell.text.strip() for cell in row.cells]
        if any(cells):
            blocks.append(" | ".join(cells))
```

Headers and footers are skipped intentionally: they repeat on every page, so including them would duplicate the same lines many times. Because python-docx has no layout engine, a DOCX's page count is unknowable, and the metadata reports `page_count: null` rather than guessing.

TXT extraction attempts a sequence of encodings — `utf-8-sig` (which also strips a byte-order mark), then `cp1252`, then `latin-1` (which maps every byte and therefore always succeeds). When the non-UTF-8 path is taken, a warning is attached: *"The file was not valid UTF-8 and was decoded as cp1252; some characters may be misread."*

5.2.3 Filename Safety and Size Limits

The uploaded filename is reduced to a safe basename before use. Directory components are stripped, so `../../etc/passwd` becomes `passwd`, and any character outside `[A-Za-z0-9._-]` is replaced:

```
_UNSAFE = re.compile(r"[^A-Za-z0-9._-]")

name = raw_name.replace("\\", "/").split("/")[-1].strip()
name = _UNSAFE.sub("_", name).lstrip(".")
```

The result is used **for display only**; files on disk are named after a generated UUID, so a hostile filename cannot influence any path the application writes to. The size limit is applied to the bytes actually received rather than to a client-supplied header, so an upload that under-reports its length is still rejected — with HTTP 413 and a message that names the configuration variable to raise.

5.3 Text Cleaning

Extraction returns what the file literally contains, which is not what a summarizer should read: a PDF page is a grid of lines, so every wrapped line ends in a meaningless newline; a word broken across two lines keeps its hyphen; running headers, footers and page numbers repeat on every page; and a single character can arrive as several code points (ligatures, curly quotes, non-breaking spaces).

Table 5.1 — Cleaning operations and their effects

Step	Effect
Character normalisation	Ligatures, curly quotes, en/em dashes and non-breaking spaces are folded to ASCII, so a generated summary compares equal to a reference written with plain characters.
Page-number removal	Forms such as `Page 12`, `12 of 30` and `- 12 -` are removed. A *lone* bare number is deliberately **kept**, because it is more likely a year or a table cell than a page number.
Header/footer removal	A short line repeating `HEADER_MIN_REPEATS` (default 3) times is boilerplate and removed. Only lines shorter than `HEADER_MAX_CHARS` (120) are ever considered, so a repeated *sentence* is never removed however often it recurs.
De-hyphenation	`summa-` ⏎ `rization` becomes `summarization`. A hyphen before a capital is preserved, as in `State-Of-The-Art`.
Reflow	Wrapped lines are re-joined into paragraphs; blank lines remain as paragraph breaks.

The module's guiding principle is stated in its own documentation: *a line is only removed when it is structurally meaningless or provably repeated, because deleting real content is a far worse failure than leaving a stray footer behind.* Non-ASCII characters are written with `chr()` in the source rather than as literals, because several of them are invisible or indistinguishable from ASCII in an editor and would be a trap for the next person to modify the file.

The cleaning stage reports what it changed — `removed_line_count`, `hyphenation_joins`, `original_character_count` — so its effect is **visible rather than silent**. The upload response exposes these counts, letting the user confirm that the document was read correctly instead of silently mangled.

5.4 Sentence Segmentation and Chunking

5.4.1 Sentence Segmentation

Sentence segmentation uses NLTK's Punkt model when its data is present and a regex splitter when it is not. Punkt is downloaded separately from the library, so — exactly like the Tesseract binary — a missing model must degrade rather than crash. The probe actually tokenizes a string rather than only checking that data files exist, because NLTK 3.9 moved to `punkt_tab` and a machine can easily have the older `punkt` present but not the newer one.

The regex fallback is deliberately conservative. It breaks after terminal punctuation except where the preceding token is a known abbreviation, an initial or part of a dotted form; decimals are safe automatically, because a full stop inside `3.14` is not followed by whitespace and so never matches the boundary pattern.

Punkt itself requires **repair**. Its English model is unsupervised and its abbreviation set excludes common forms: given "as shown in Fig. 3" it splits after "Fig.", and "reduce this risk, i.e. they select" after "i.e.". Such fragments are actively harmful downstream — an extractive summarizer can select "as shown in Fig." as a summary sentence — so `_repair_splits` re-joins segments ending in a known abbreviation, a dotted form, an initial, or an unclosed bracket. Repeated merging is intentional, since "J. R. R. Tolkien" can arrive as several fragments and each merge may reveal another abbreviation at the new end.

5.4.2 Chunking

`chunk_text` splits a document into sentence-aligned, overlapping chunks. Two decisions shape it:

- **Split on sentences, never mid-sentence.** A chunk that begins or ends mid-sentence hands the summarizer a fragment, and fragments produce garbled output.
- **Overlap consecutive chunks.** Without overlap, a fact spanning a boundary can be split across two chunks and survive in neither.

The overlap is measured in characters but applied on **sentence boundaries**, so it never cuts a sentence either:

```
def _overlap_tail(sentences: list[str], budget: int) -> list[str]:
    if budget <= 0 or not sentences:
        return []

    taken = [sentences[-1]]
    total = len(sentences[-1])

    for sentence in reversed(sentences[:-1]):
        cost = len(sentence) + 1  # + the joining space
        if total + cost > budget:
            break
        taken.insert(0, sentence)
        total += cost

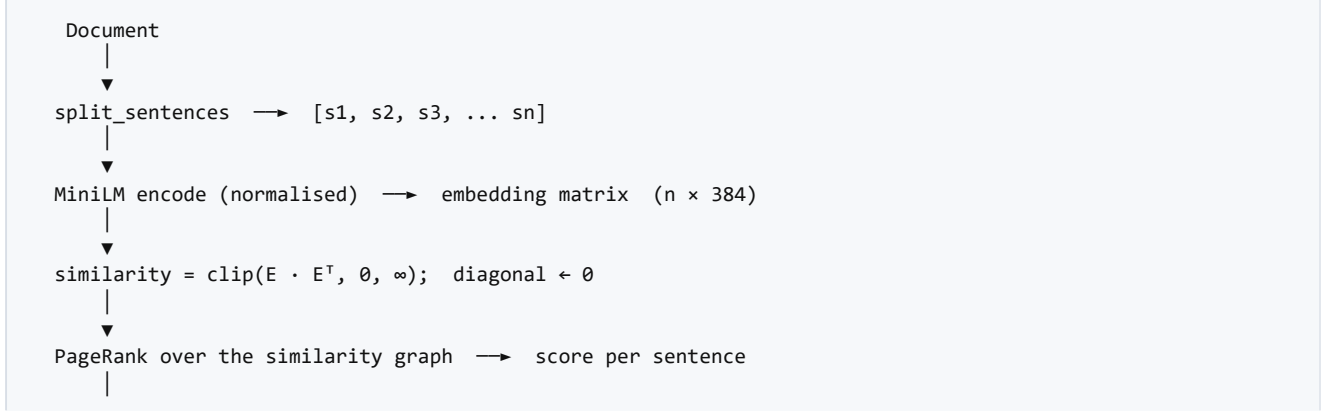
    return taken
```

Note that the final sentence is **always** carried when any overlap is requested, even if it alone exceeds the budget. The purpose of an overlap is to bridge the boundary between two chunks, and a sentence that stops short of the boundary bridges nothing: skipping past it to shorter, older sentences would duplicate text already wholly contained in the previous chunk while leaving the seam bare. `budget` is therefore a target, not a hard cap.

One further edge case is handled: a single "sentence" longer than the chunk size — a table row, or a mangled OCR line with no punctuation — is broken on **word** boundaries by `_split_oversized`, since otherwise it would produce a chunk no model could accept. Overlap is also capped at `size - 1` so that chunking always advances; a chunk consisting entirely of overlap would carry nothing forward.

5.5 Extractive Summarization

Figure 5.1 — Extractive summarization flow



```
    ▼
keep top-k by score  ──►  restore original document order
    │
    ▼
 Summary (verbatim sentences)
```

The implementation is compact because the mathematical steps compose directly:

```
embeddings = _model().encode(sentences, normalize_embeddings=True)

# Normalised embeddings make the dot product the cosine similarity.
# Similarities are clipped at zero because PageRank needs non-negative
# weights, and the diagonal is cleared so a sentence cannot reinforce itself.
similarity = np.clip(embeddings @ embeddings.T, 0.0, None)
np.fill_diagonal(similarity, 0.0)

scores = nx.pagerank(nx.from_numpy_array(similarity))
ranked_indices = sorted(scores, key=scores.get, reverse=True)
```

Three details are worth drawing out. The **normalisation** at encode time turns the matrix product into a cosine-similarity matrix in a single operation. The **clipping at zero** is required because PageRank weights must be non-negative. Clearing the **diagonal** prevents a sentence from reinforcing its own score, which would otherwise inflate the rank of every sentence uniformly.

The proportion of sentences retained is set by the requested length:

```
LENGTH_RATIOS = {"short": 0.15, "medium": 0.3, "long": 0.5}
```

Selection then **restores the original document order**: the top-*k* ranked sentences are chosen, but the summary is assembled by walking the original sentence list and taking those selected. A summary whose sentences appeared in rank order rather than document order would read as a jumble. The implementation also counts occurrences rather than testing set membership, so a sentence that legitimately appears twice in the source is not silently duplicated or dropped.

Two guards exist: a document with fewer than `MIN_SENTENCES` (3) sentences is returned unchanged, because compressing three sentences into one loses more than it gains; and a single-sentence document is returned as-is.

The factual guarantee is structural, not statistical. No sentence is ever rewritten or generated — every sentence in the output is a verbatim copy of a sentence in the input. An extractive summary therefore *cannot* contain a fact the source did not state. This is the property that makes the mode appropriate for legal, financial or medical documents where exact wording matters.

5.6 Abstractive Summarization

Figure 5.2 — Abstractive map-reduce flow

```
  Document
    │
    ▼
tokens = tokenizer.encode(text)
    │
    ├── tokens ≤ 900 ──────────────►  BART generate  ──►  Summary
    │
```

```
    └── tokens > 900
            │
            ▼
      MAP: chunk_text(document)
            │
            ▼
      BART generate per chunk  ──►  [partial_1, partial_2, ... partial_n]
            │
            ▼
      joined = " ".join(partials)
            │
            ├── joined ≤ 900 tokens ──►  Summary
            │
            └── joined > 900 tokens
                    │
                    ▼
             REDUCE: chunk and summarize once more  ──►  Summary
```

The control flow is:

```
def summarize(text: str, length: str = "medium") -> str:
    stripped = text.strip()
    words = stripped.split()
    if len(words) < MIN_WORDS:          # 40 words
        return stripped

    tokenizer, _ = _model()
    token_ids = tokenizer.encode(stripped, truncation=False)

    if len(token_ids) <= MAX_SINGLE_PASS_TOKENS:      # 900
        return _generate_chunk_summary(stripped, length)

    chunks = chunking_service.chunk_text(stripped)
    partial_summaries = [_generate_chunk_summary(c.text, length) for c in chunks]
    joined_summary = " ".join(partial_summaries).strip()

    joined_tokens = tokenizer.encode(joined_summary, truncation=False)
    if len(joined_tokens) > MAX_SINGLE_PASS_TOKENS:
        reduce_chunks = chunking_service.chunk_text(joined_summary)
        if reduce_chunks:
            reduced = [_generate_chunk_summary(rc.text, length) for rc in reduce_chunks]
            return " ".join(reduced).strip()

    return joined_summary
```

The **reduce step runs at most once**, deliberately. Looping until the output fits would be non-terminating in principle and unbounded in cost; a single condensation is the documented policy. The 900-token single-pass cap is set below BART's hard 1024-token limit to leave margin for the decoder and for tokenizer boundary effects.

Generation parameters vary with the requested length:

Table 5.2 — Summary length parameters

Length	`max_new_tokens`	`min_new_tokens`
Short	120	40
Medium	220	80
Long	400	140

Beam search is used with `num_beams=4` and `early_stopping=True`, which trades some speed for noticeably more fluent output than greedy decoding. One subtlety is handled in `_generate_chunk_summary`: when the input is itself very short, the requested `min_new_tokens` would force the model to generate more tokens than the input contains, producing fabricated padding. The minimum is therefore clamped:

```
min_tokens = min(params["min_new_tokens"], max(10, input_token_count - 5))
max_tokens = max(min_tokens + 10, params["max_new_tokens"])
```

Documents under `MIN_WORDS` (40 words) are returned unchanged, since asking a summarizer to condense a paragraph-length input invites the model to expand rather than compress.

5.7 Hybrid Summarization

Figure 5.3 — Hybrid summarization flow

```
   Document
      │
      ▼
  split_sentences + rank_sentences (TextRank)
      │
      ▼
  keep top 50% by rank  ──►  restore original document order
      │
      ▼
  filtered_text  (only high-importance sentences)
      │
      ▼
  abstractive_service.summarize(filtered_text, length)
      │
      ▼
   Summary (generated, constrained to central content)
```

The implementation is a short composition of the two preceding services, which is the point: the hybrid strategy introduces no new algorithm, only a different order of operations.

```
def summarize(text: str, length: str = "medium") -> str:
    sentences = chunking_service.split_sentences(text)
    if len(sentences) < MIN_SENTENCES:
        return text.strip()

    ranked = extractive_service.rank_sentences(text)

    keep_count = max(1, round(len(sentences) * 0.5))
    top_half = ranked[:keep_count]
    # ... restore original document order into `filtered_text` ...

    return abstractive_service.summarize(filtered_text, length=length)
```

Why verbatim selection before generation matters. Feeding BART the full document gives it the freedom to paraphrase *any* sentence, including unimportant or tangential ones, and every paraphrase is an opportunity to assert something the source did not say. Feeding it only the Top-50% sentences by TextRank restricts the generation to content the graph algorithm identified as central to the document, and excludes the remaining half entirely. The result is a summary that reads fluently — it is still generated text — but whose factual surface is substantially smaller.

The trade-off is honest and worth stating: the filter is a **recall risk as well as a fidelity benefit**. If the ranker ranks a genuinely important sentence below the 50% threshold, the abstractive stage can never recover it, because that sentence was never passed to the model.

5.8 Evaluation Module

The evaluation service is intentionally thin, because the metrics are computed by the standard packages rather than reimplemented — reimplementation would risk disagreeing with published results.

```
def evaluate(summary: str, reference: str) -> dict[str, float]:
    sum_text, ref_text = summary.strip(), reference.strip()
    if not sum_text or not ref_text:
        return {"rouge_1": 0.0, "rouge_2": 0.0, "rouge_l": 0.0, "bleu": 0.0}

    scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"], use_stemmer=True)
    rouge_results = scorer.score(ref_text, sum_text)
    bleu_result = sacrebleu.corpus_bleu([sum_text], [[ref_text]])

    return {
        "rouge_1": float(rouge_results["rouge1"].fmeasure),
        "rouge_2": float(rouge_results["rouge2"].fmeasure),
        "rouge_l": float(rouge_results["rougeL"].fmeasure),
        "bleu": float(bleu_result.score),
    }
```

Three implementation points:

- **Stemming is enabled** (`use_stemmer=True`) so that "summarizing" and "summarize" count as a match. Without it, ROUGE would penalise morphological variation that carries no difference in meaning.
- **Numeric types are cast to `float`** explicitly. The `rouge-score` package returns NumPy floats, which the JSON serialiser cannot encode; the cast is what prevents a serialisation failure at the API boundary.
- **Empty input short-circuits** to an all-zero result rather than raising. Supplying an empty reference is a user input error, not a server fault, and the frontend renders zeros sensibly.

The distinction between the two metric families is the reason both are reported. ROUGE is recall-oriented — "how much of the reference did the summary capture?" — while BLEU is precision-oriented with a brevity penalty — "how much of the summary is correct, and is the summary long enough to be worth anything?" A very short summary can score respectably on BLEU's per-gram precision but is penalised by the brevity penalty and scores poorly on ROUGE recall.

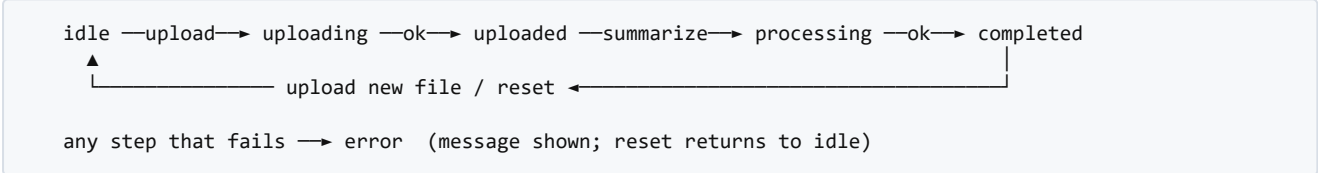
5.9 Frontend Implementation

The frontend is a React single-page application built with Vite, styled with Tailwind CSS v4 and communicating with the backend through axios. It is deliberately a **small, flat structure**: one page, six presentational components and a single API module.

File	Responsibility
`src/pages/Home.jsx`	The page: owns all state, drives the workflow, composes the components
`src/services/api.js`	The only module that knows about HTTP: base URL, timeout, the four calls, error normalisation
`src/components/FileUploader.jsx`	Drag-and-drop / file-picker upload control
`src/components/ModeSelector.jsx`	Extractive / Abstractive / Hybrid selection
`src/components/LengthSelector.jsx`	Short / Medium / Long selection
`src/components/SummaryDisplay.jsx`	Renders the generated summary
`src/components/EvaluationPanel.jsx`	Reference-summary input and metric display
`src/components/Navbar.jsx`	Application header

5.9.1 Page State Machine

Figure 5.4 — Frontend page state machine



The page is driven by a single state variable with six values, which keeps the rendering logic declarative: the interface shows what the current state permits and nothing else.

```
// 'idle' | 'uploading' | 'uploaded' | 'processing' | 'completed' | 'error'
const [pageState, setPageState] = useState('idle')
```

The controls panel only appears once `uploadResult` is populated; the summary panel only appears once `summaryResult` is; the evaluation panel appears alongside the summary. This avoids a spaghetti of booleans and guarantees that the user is never invited to summarize a document that has not been uploaded.

5.9.2 API Module and Error Normalisation

All HTTP knowledge is confined to `src/services/api.js`. Components never build URLs or touch axios directly, so swapping axios for `fetch`, or changing the base URL, touches exactly one file.

Its most valuable function is `normaliseError`, which collapses every possible failure mode into a single `Error` with a readable `message`. An axios interceptor applies it to every response:

```
apiClient.interceptors.response.use(
  (response) => response,
  (error) => Promise.reject(normaliseError(error)),
)
```

Four failure modes are distinguished and given *useful* messages rather than raw status codes:

Condition	Message shown to the user
HTTP error response (400/413/422/500)	The backend's `detail` string, which is already written for an end user
Timeout	"The request timed out. Very long documents can take several minutes on CPU — try a shorter document or the Extractive mode."
Server unreachable	"Cannot reach the backend at `<base URL>`. Make sure the FastAPI server is running."
Unexpected	The underlying error message, or a generic fallback

Because normalisation happens once, in one place, every component can render `error.message` without defensive checks. It also handles FastAPI's list-shaped validation errors, joining them into one string, even though the backend already flattens them — belt and braces at a boundary where a mismatch would otherwise surface as `[object Object]`.

5.9.3 Connection Probe and Generous Timeout

On load the page probes `GET /api/health` and renders one of three states: a "checking" indicator, a green connection, or an explanatory failure panel that shows the exact command needed to start the backend and offers a retry button. The purpose is to fail early and legibly: without the probe, a user whose backend is not running would only discover it after waiting for an upload to fail.

The axios client's timeout is set to **300 seconds**:

```
const apiClient = axios.create({
  baseURL: API_BASE_URL,
  timeout: 300_000,
})
```

This is not arbitrary. Summarizing a long document on CPU with the abstractive or hybrid mode involves map-reduce over many chunks, each requiring beam-search generation, and can genuinely take minutes. A conventional thirty-second timeout would abort requests that were about to succeed.

CHAPTER 6 — TESTING

6.1 Testing Strategy

Testing was treated as part of each phase rather than a final activity. The backend's `api/` – `services/` separation exists partly to make this possible: because every service is a plain function over strings, most of the logic is exercised without an HTTP server, an upload, or a browser.

Three levels are used:

1. **Unit tests** — the largest group. Each service is tested directly: extraction from each of the four input kinds, each cleaning operation, the regex splitter, the Punkt repair function, chunk boundaries and overlap, and each summarizer's guards.
2. **API tests** — the four endpoints are exercised through FastAPI's test client (`httpx`), covering both success paths and error paths (unsupported type, oversized file, empty file, unknown document id).
3. **Manual and interactive testing** — the running application was exercised in the browser with real PDFs, DOCX files, TXT files and scanned documents, which is how the OCR path and the frontend state machine were verified end to end.

Test fixtures are generated by a script (`tests/make_fixtures.py`) that produces a sample PDF, DOCX and TXT plus a deliberately scanned PDF and an empty file, so the suite is self-contained and does not depend on binary assets being committed by hand.

6.2 Test Suite Composition

The suite comprises **89 automated tests** across nine test modules.

Table 6.1 — Test suite composition by module

Test module	Tests	Focus
`test_chunking_service.py`	24	Sentence segmentation, the regex fallback, Punkt split repair, chunk boundaries, overlap behaviour, oversized sentences
`test_cleaning_service.py`	23	Character normalisation, page-number removal, header/footer detection, de-hyphenation, reflow, conservatism guards
`test_document_service.py`	16	Extraction for PDF, DOCX, TXT and scanned PDF; OCR orchestration; storage round-trip; metadata
`test_summarize.py`	8	The summarization endpoint and mode dispatch
`test_api.py`	7	Upload endpoint, extension rejection, size limit, empty file, health endpoint
`test_evaluation_service.py`	5	ROUGE-1/2/L and BLEU computation, identical input, empty input, numeric types
`test_abstractive_service.py`	3	Short-input guard, single-pass path, map-reduce dispatch
`test_hybrid_service.py`	3	Short-input guard, filtering to the top 50%, delegation to the abstractive stage
Total	**89**	

Running the suite:

```
cd backend
.venv/Scripts/python.exe tests/make_fixtures.py   # once: generates tests/fixtures/
.venv/Scripts/python.exe -m pytest tests -v
```

6.3 Techniques for Portability

Two external dependencies are optional, and a test suite that assumed either was installed would fail on a machine that lacked it. Both are therefore isolated behind seams.

OCR orchestration is tested with the recognition step stubbed. The logic that matters — which pages are rendered, how results are merged back into the page list, what happens when Tesseract fails part-way through a document, and what the user is told — is tested with the recognition call stubbed out, so it runs on **every** machine. Only the two tests that assert real recognition *quality* skip when Tesseract is absent. This is a deliberate trade: the orchestration is the code this project wrote, and it should be tested everywhere; recognition accuracy belongs to Tesseract.

Punkt's split behaviour is tested through the repair function. Rather than requiring the NLTK data to be present, the tests construct synthetic Punkt-style output — including the bad splits Punkt is known to produce, such as a break after "Fig." — and call `_repair_splits` on it. The assertions therefore hold whether or not the data is installed.

A third seam exists for the models themselves: because both models are loaded through `@lru_cache`-decorated getters called *inside* the functions that need them, the guards in each summarizer (the short-input checks, the token-count branches) can be tested without downloading or loading either model.

6.4 Sample Test Cases

Table 6.2 — Representative test cases

#	Test case	Input	Expected result
1	Unsupported file type is rejected	A `.png` file	HTTP 400, message listing the allowed extensions
2	Oversized file is rejected	A 25 MB TXT file	HTTP 413, message naming `MAX_FILE_SIZE_MB`
3	Empty file is rejected	A zero-byte TXT	HTTP 400, "The uploaded file is empty."
4	TXT in a legacy encoding is read	TXT encoded as cp1252	Text extracted, warning that it was decoded as cp1252
5	DOCX tables are extracted	DOCX with a table	Table cells present in the extracted text, joined with `|`
6	Scanned PDF without Tesseract	Image-only PDF	Upload rejected with a message naming the fix; `ocr.available` is `false`
7	Scanned PDF with Tesseract	Image-only PDF	Text recovered; `ocr_used` is `true`; a warning names the OCR pages
8	Partly scanned PDF	3 digital pages + 1 image page	Digital pages kept as-is; only the image page is OCR'd
9	Page number is removed	`Page 12 of 30` on its own line	Line absent from cleaned text; `removed_line_count` incremented
10	A lone bare number is kept	A line containing only `2024`	Line retained (could be a year or a table cell)
11	Repeated short header is removed	`Internal Report` repeated 3×	Line removed
12	A repeated full sentence is kept	A long sentence repeated 3×	Line retained — a repeated sentence is content, not a header
13	Hyphenated line break is repaired	`summa-` ⏎ `rization`	`summarization`; `hyphenation_joins` is 1
14	A capitalised hyphen is preserved	`State-Of-The-Art`	Unchanged
15	Decimals do not split sentences	`The value is 3.14 exactly.`	One sentence, not two
16	An abbreviation does not split	`as shown in Fig. 3`	Re-joined into a single sentence
17	Chunk overlap never cuts a sentence	A long document	Every chunk ends on a sentence boundary
18	Every sentence appears in a chunk	A long document	Union of chunks covers all sentences; nothing dropped
19	Extractive output is verbatim	Any document	Every sentence in the summary appears in the source unchanged

#	Test case	Input	Expected result
20	Extractive short document is unchanged	A 2-sentence document	Returned as-is
21	Abstractive short input is unchanged	A 30-word document	Returned as-is (below the 40-word guard)
22	Identical summary and reference score 1.0	Summary == reference	`rouge_1 = rouge_2 = rouge_l = 1.0`
23	Empty reference yields zeros	Empty reference string	All four metrics are `0.0`, no exception
24	Metrics serialise to JSON	Any evaluation	Response contains plain floats, not NumPy types
25	Unknown document id returns 404	`POST /api/summarize` with a random id	HTTP 404, "Document not found or no longer available..."
26	Invalid mode is rejected at the boundary	`mode="extractiv"`	HTTP 422 before any service is called
27	Health reports OCR availability	`GET /api/health`	Response contains `ocr.available` as a boolean

---

CHAPTER 7 — RESULTS AND DISCUSSION

7.1 Functional Results

All twelve planned development phases except containerised deployment were completed. The system delivers, verified end to end in the browser:

Capability	Result
PDF ingestion (digital)	Text layer extracted with PyMuPDF; page count and extraction method reported
PDF ingestion (scanned)	Image pages detected and routed to Tesseract; `ocr_used` true; per-page warnings reported
PDF ingestion (partly scanned)	Digital pages retained, image pages filled in; hybrid extraction method reported
DOCX ingestion	Paragraphs **and** tables extracted; page count reported as unknown rather than guessed
TXT ingestion	Encoding detected (UTF-8 / cp1252 / latin-1); a warning issued when not UTF-8
Text cleaning	Page numbers and repeated headers removed, hyphenated breaks repaired, prose reflowed; counts surfaced to the UI
Extractive summarization	Three length settings; output sentences verbatim; original document order preserved
Abstractive summarization	BART generation; single-pass for short documents, map-reduce for long ones
Hybrid summarization	Top-50% TextRank filtering followed by BART condensation
Evaluation	ROUGE-1, ROUGE-2, ROUGE-L (0.0–1.0) and BLEU (0–100) computed against a supplied reference
Error handling	Uniform JSON error shape; validation errors flattened; no stack traces leak to the client
Automated tests	89 tests passing

7.2 Behaviour of the Three Strategies

Observed behaviour matches the design intent, and the differences between the three modes are clearly visible on the same document.

Extractive. Output is a set of complete sentences lifted directly from the source. It reads as a collection of the document's load-bearing statements rather than as continuous prose, which is expected: sentences are selected for centrality, not for how they follow one another. Its indispensable property is that a generated sentence can be checked against the source by search — if it is in the summary, it is in the document, word for word.

Abstractive. Output reads as continuous, human-like prose and compresses more aggressively than the extractive mode. It is also the mode where the user should read critically: the model can state something plausible and consistent with the surrounding text that the source does not in fact say.

Hybrid. Output retains the fluency of the abstractive mode, but inspection shows it tracks the extractively selected sentences closely — it paraphrases central content rather than ranging over the whole document. This is the intended effect of constraining generation, and it is the mode that strikes the most useful balance for general reading.

7.3 Performance Observations

- **Model loading dominates the first request.** The first summarization call in a process pays for loading the selected model — roughly 90 MB for MiniLM and roughly 1.6 GB for BART. Subsequent calls reuse the cached instance. This is the reason for the `@lru_cache` singleton design (§4.6); reloading per request would add roughly ten seconds to every call.
- **Extractive mode is markedly faster than either generative mode**, since it performs one embedding pass plus a PageRank computation and generates no text.
- **Abstractive cost scales with document length**, roughly linearly in the number of chunks: each chunk requires a beam-search generation, and a document ten times longer produces roughly ten times as many chunk generations. This is why the frontend allows a five-minute timeout and why the timeout message suggests the extractive mode as an alternative.
- **The event loop stays responsive during inference.** Because blocking work is dispatched to a worker thread, `/api/health` answers normally while a long document is being summarized. This was verified by polling the health endpoint during an abstractive run.
- **Startup is cheap.** Because the heavy imports (`torch`, `transformers`) occur inside the cached getters rather than at module import, the API server starts in well under a second and does not occupy memory with PyTorch until a summarization is actually requested.
- **OCR cost is paid only where it is needed.** Pages with a text layer are never rendered, so a digital PDF pays nothing for the OCR capability; a scanned document pays the render-and-recognise cost per image page at 200 DPI.

7.4 Limitations

The following limitations are known, characterised, and in most cases the result of a deliberate conservative choice.

1. **Abstractive hallucination.** The language model can generate statements not present in the source. This is inherent to abstractive summarization and is the reason the hybrid mode exists — and the reason the extractive mode is offered as the zero-risk option.

2. **OCR accuracy depends on scan quality.** A summary of a scanned document inherits Tesseract's recognition errors. Extracted text is deliberately **not** corrected or spell-checked, because guessing at what a mangled word "should" be risks introducing content the document does not contain — a worse failure than a visible typo.

3. **Header/footer removal only catches exact repeats.** A footer whose text changes on every page (`Report 2024 - page 3`) survives, because masking the digits to catch it would also delete genuinely

numbered short lines such as a run of table captions. Removing a real line is worse than keeping a footer.

4. **Sentence segmentation is heuristic.** Common abbreviations, initials, decimals and dotted forms are handled, but an unusual abbreviation can still be mistaken for a sentence end. This matters more in the extractive mode than elsewhere, because extractive summarization quotes sentences verbatim — a bad split shows up directly in the output rather than being smoothed over.
5. **ROUGE and BLEU are reference-based.** They cannot be computed without a human-written reference, so evaluation is optional and user-supplied. When no reference is provided, no quality score is available.
6. **Metrics penalise valid paraphrase.** Both metrics are based on n-gram overlap. An abstractive summary that correctly uses a synonym where the reference used another word is scored lower despite being accurate. This is a known property of surface-overlap metrics, and it means abstractive summaries typically score lower than extractive ones against the *same* reference even when they read better.
7. **Transformer inference runs on CPU by default.** Long documents take minutes. The device is configurable (`ABSTRACTIVE_DEVICE`) but GPU support was not exercised in this project.
8. **The document store is process-local and not persistent.** Files in `uploads/` are the only record of an uploaded document. Clearing that directory makes existing `document_id` values return 404. There is also no cleanup of old uploads and no summary history.
9. **Single language.** OCR and both pretrained models are configured for English. Other languages would require different language packs and different models.
10. **No containerised or cloud deployment.** Docker packaging was deferred to a later phase and not completed; the system currently runs as two local development processes.

CHAPTER 8 — CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project set out to build a system that takes an arbitrary long-form document — possibly scanned, possibly poorly formatted, possibly far longer than any summarization model can read in one pass — and returns a concise summary, while giving the user control over the trade-off between factual fidelity and linguistic fluency.

That objective has been met. The completed system:

- ingests **PDF, DOCX and TXT** documents, extracting paragraphs and tables from DOCX, detecting text encodings in TXT, and — for PDFs — routing pages with no text layer to **Tesseract OCR** while leaving digital pages untouched;
- **cleans** the extracted text into flowing prose, removing page numbers and repeated headers and repairing hyphenated line breaks, while deliberately erring toward keeping content;
- **chunks** documents on sentence boundaries with overlap, so that documents exceeding the model's 1024-token limit are summarized in full rather than silently truncated;
- implements **three summarization strategies** — extractive (embeddings + TextRank, structurally incapable of hallucination), abstractive (BART with map-reduce), and hybrid (TextRank filtering before BART condensation);
- **evaluates** any generated summary against a reference using ROUGE-1, ROUGE-2, ROUGE-L and BLEU;
- exposes all of this through a **typed REST API** and a responsive **single-page web interface**;
- and is validated by **89 automated tests**.

Several findings seem to us more generally useful than the application itself. First, **the pipeline matters as much as the model**: the quality of the summary depended at least as much on correct extraction, cleaning and chunking as on the choice of summarization model, and most of the engineering effort went into those stages. Second, **offering more than one strategy is not hedging** — the extractive, abstractive and hybrid modes fail in genuinely different ways, and which one is appropriate depends on the document and on the cost of being wrong. Third, **graceful degradation is a design decision, not an afterthought**: treating both Tesseract and the NLTK data as optional, and reporting their absence as data rather than as errors, is what makes the application installable and demonstrable on any machine rather than only on the machine it was developed on.

8.2 Future Scope

The following extensions are natural next steps, listed roughly in order of value relative to effort.

1. **Persistent storage and summary history.** Replace the file-based document store with SQLite or PostgreSQL so that documents and generated summaries survive a restart, can be listed, and can be compared side by side across modes and lengths. This would also enable the batch evaluation experiments that the current design makes awkward.

2. **GPU inference and quantised models.** Move inference to a GPU, or use a quantised BART variant, to cut the summary time for long documents from minutes to seconds. The `ABSTRACTIVE_DEVICE` setting already exists; what is missing is testing on GPU hardware and an evaluation of the quality cost of quantisation.
3. **Multi-document and query-focused summarization.** Summarize several documents into one coherent summary, or summarize with respect to a user's question rather than generically. The chunking and map-reduce machinery already generalises to the first of these.
4. **Multilingual support.** Language packs for Tesseract and multilingual or per-language summarization models, together with language detection for automatic model selection.
5. **Export formats.** DOCX and PDF export alongside the current plain text, with the summary, the source metadata and the evaluation scores presented as a single document.
6. **Containerised deployment.** A Docker Compose configuration bundling the backend (with Tesseract and the model cache as build layers) and the frontend, so the system can be deployed with one command.
7. **A richer evaluation mode with multiple references.** To date, evaluation accepts a single reference. Supporting several references per document would reduce the sensitivity of the scores to one annotator's phrasing, which is the main source of noise in reference-based metrics.
8. **Streaming progress for long documents.** Report map-reduce progress to the client as chunks are summarized, replacing the current indeterminate spinner with a determinate progress indicator.
9. **Faster sentence-similarity computation.** The extractive stage computes a full pairwise similarity matrix, which is quadratic in the number of sentences. Approximate nearest-neighbour search would make very long documents substantially faster.

REFERENCES

1. H. P. Luhn, "The Automatic Creation of Literature Abstracts," *IBM Journal of Research and Development*, vol. 2, no. 2, pp. 159–165, 1958.
2. R. Mihalcea and P. Tarau, "TextRank: Bringing Order into Texts," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2004, pp. 404–411.
3. L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab Technical Report, 1999.
4. A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
5. M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proceedings of ACL*, 2020.
6. C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004, pp. 74–81.
7. K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 311–318.
8. I. Mani and M. T. Maybury (eds.), *Advances in Automatic Text Summarization*, MIT Press, 1999.
9. H. Edmundson, "New Methods in Automatic Extracting," *Journal of the ACM*, vol. 16, no. 2, pp. 264–285, 1969.
10. R. Smith, "An Overview of the Tesseract OCR Engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
11. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.
12. T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of EMNLP: System Demonstrations*, 2020, pp. 38–45.
13. FastAPI documentation — https://fastapi.tiangolo.com
14. Pydantic documentation — https://docs.pydantic.dev
15. React documentation — https://react.dev
16. Vite documentation — https://vite.dev
17. PyMuPDF documentation — https://pymupdf.readthedocs.io
18. Tesseract OCR (UB Mannheim Windows builds) — https://github.com/UB-Mannheim/tesseract/wiki
19. `rouge-score` — https://github.com/google-research/google-research/tree/master/rouge

20. `sacrebleu` — https://github.com/mjpost/sacrebleu

21. NetworkX documentation — https://networkx.org/documentation/stable/

22. NLTK documentation — https://www.nltk.org

APPENDIX A — API ENDPOINT REFERENCE

A.1 `GET /api/health`

Liveness and readiness probe, called by the frontend on page load.

Response (200)

```
{
  "status": "ok",
  "service": "AI Document Summarization API",
  "version": "0.1.0",
  "python_version": "3.11.x",
  "platform": "Windows",
  "timestamp": "2026-01-01T00:00:00+00:00",
  "ocr": { "available": true, "language": "eng" }
}
```

When Tesseract is not usable, `ocr.available` is `false` and a `reason` field explains why. A health endpoint must not be the thing that fails, so any error while probing Tesseract is reported as data rather than raised — OCR is optional, and the API is healthy without it.

A.2 `POST /api/upload`

Multipart upload of a PDF, DOCX or TXT file. Returns metadata only — **never the extracted text**.

Request: `multipart/form-data` with a `file` field.

Response (200)

```
{
  "document_id": "9f2c...",
  "filename": "report.pdf",
  "file_type": "pdf",
  "character_count": 48210,
  "word_count": 7640,
  "chunk_count": 49,
  "page_count": 22,
  "ocr_used": false,
  "extraction_method": "PyMuPDF text layer",
  "cleaning": {
    "raw_character_count": 49002,
    "removed_line_count": 44,
    "hyphenation_joins": 7
  },
  "preview": "First ~280 characters of the cleaned text...",
  "warnings": []
}
```

Errors

Status	Cause
400	Unsupported extension, empty file, no filename, or no readable text
413	File exceeds `MAX_FILE_SIZE_MB`
422	File could not be parsed (corrupt, password-protected, or an image-only PDF with no OCR available)

`chunk_count` reports how many chunks the document would be split into at the current `CHUNK_SIZE`, so the user can anticipate the cost of summarizing it.

A.3 `POST /api/summarize`

Request

```
{
  "document_id": "9f2c...",
  "mode": "extractive",
  "length": "medium"
}
```

`mode` is one of `extractive`, `abstractive`, `hybrid`; `length` is one of `short`, `medium`, `long`.

Response (200)

```
{
  "document_id": "9f2c...",
  "mode": "extractive",
  "length": "medium",
  "summary": "The generated summary text."
}
```

Errors: 404 if the `document_id` is unknown; 422 if `mode` or `length` is invalid.

A.4 `POST /api/evaluate`

Summarizes the document and scores the result against a supplied reference.

Request

```
{
  "document_id": "9f2c...",
  "reference": "A human-written summary of the same document.",
  "mode": "extractive",
  "length": "medium"
}
```

Response (200)

```
{
  "document_id": "9f2c...",
  "mode": "extractive",
  "length": "medium",
  "summary": "The generated summary text.",
  "reference": "A human-written summary of the same document.",
  "metrics": {
```

```
    "rouge_1": 0.482,
    "rouge_2": 0.221,
    "rouge_l": 0.401,
    "bleu": 27.4
  }
}
```

ROUGE metrics are in the range 0.0–1.0; BLEU is on a 0–100 scale.

Error shape (all endpoints)

```
{ "detail": "Human-readable explanation for the user." }
```

APPENDIX B — SETUP AND EXECUTION

B.1 Prerequisites

- **Python 3.11** — verify with `py -3.11 --version`
- **Node.js 20.19+ or 22.12+** — verify with `node --version`
- **Tesseract OCR 5.x** — required only for scanned PDFs

Installing Tesseract on Windows

1. Download the installer from https://github.com/UB-Mannheim/tesseract/wiki.
2. Install to the default location: `C:\Program Files\Tesseract-OCR`.
3. Either add that folder to `PATH`, or set `TESSERACT_CMD` in `.env`:

   ```
   TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe
   ```

4. Verify with `tesseract --version`.

> The Windows installer — including the winget package — does **not** reliably add Tesseract to `PATH`. If `tesseract --version` works in a terminal but the application still reports `ocr.available: false`, set `TESSERACT_CMD` in `.env`. That is the deterministic fix, and it is what this project uses.

On Linux, `sudo apt install tesseract-ocr`; on macOS, `brew install tesseract`.

B.2 Environment Files

```
cp .env.example .env                    # root: backend settings
cp frontend/.env.example frontend/.env  # frontend: API base URL
```

Key backend settings, with their defaults:

Variable	Default	Purpose
`MAX_FILE_SIZE_MB`	20	Upload size limit
`ALLOWED_EXTENSIONS`	`.pdf,.docx,.txt`	Permitted upload types
`OCR_MIN_CHARS_PER_PAGE`	80	Page-text threshold below which a PDF page is treated as an image
`OCR_DPI`	200	Resolution used when rendering pages for OCR
`OCR_LANGUAGE`	eng	Tesseract language pack
`TESSERACT_CMD`	*(unset)*	Explicit path to the Tesseract binary
`CHUNK_SIZE`	1000	Target chunk size in characters
`CHUNK_OVERLAP`	100	Target overlap between consecutive chunks
`HEADER_MIN_REPEATS`	3	Repeats before a short line is treated as a header or footer
`EXTRACTIVE_MODEL`	`sentence-transformers/all-MiniLM-L6-v2`	Embedding model
`ABSTRACTIVE_MODEL`	`facebook/bart-large-cnn`	Summarization model
`ABSTRACTIVE_DEVICE`	cpu	Torch device for inference
`CORS_ORIGINS`	`http://localhost:5173,...`	Origins permitted to call the API

B.3 Backend Installation

```
cd backend
py -3.11 -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.txt

# One-off: NLTK's sentence-tokenizer data. `pip install nltk` ships the library
# but not the models, so this download is a separate step.
.venv/Scripts/python.exe -c "import nltk; nltk.download('punkt_tab')"
```

NLTK data is optional. Without it the pipeline falls back to a built-in regex sentence splitter that handles abbreviations and decimals. Punkt is better on complex prose, so the download is worth doing.

B.4 Frontend Installation

```
cd frontend
npm install
```

B.5 Running the Project

Two terminals are required.

Terminal 1 — backend

```
cd backend
.venv/Scripts/python.exe -m uvicorn app.main:app --reload --port 8000
```

Terminal 2 — frontend

```
cd frontend
npm run dev
```

Open http://localhost:5173. The home page probes `GET /api/health` and shows a green "Backend connected" panel when the two halves are communicating. Interactive API documentation is available at http://localhost:8000/docs.

> **Heads-up:** the first run downloads model weights — approximately 90 MB for `all-MiniLM-L6-v2` and approximately 1.6 GB for `facebook/bart-large-cnn`. They are cached locally afterwards and the download is not repeated.

B.6 Running the Tests

```
cd backend
.venv/Scripts/python.exe tests/make_fixtures.py   # once: generates tests/fixtures/
.venv/Scripts/python.exe -m pytest tests -v
```

B.7 Project Structure

```
ai-document-summarization/
├── backend/
│   ├── app/
│   │   ├── main.py              # application factory, CORS, error handlers
│   │   ├── config.py            # settings from environment / .env
│   │   ├── api/                 # HTTP endpoints
│   │   │   ├── health.py
│   │   │   ├── upload.py
│   │   │   ├── summarize.py
│   │   │   └── evaluate.py
│   │   ├── services/            # business logic
│   │   │   ├── document_service.py
│   │   │   ├── ocr_service.py
│   │   │   ├── cleaning_service.py
│   │   │   ├── chunking_service.py
│   │   │   ├── extractive_service.py
│   │   │   ├── abstractive_service.py
│   │   │   ├── hybrid_service.py
│   │   │   └── evaluation_service.py
│   │   ├── models/schemas.py    # request / response contracts
│   │   └── utils/               # filename and text helpers
│   ├── tests/                   # 89 automated tests
│   ├── requirements.txt
│   └── README.md
├── frontend/
│   ├── src/
│   │   ├── components/          # FileUploader, ModeSelector, LengthSelector,
│   │   │                        # SummaryDisplay, EvaluationPanel, Navbar
│   │   ├── pages/Home.jsx       # the single page
│   │   ├── services/api.js      # the only module that knows about HTTP
│   │   ├── App.jsx
```

```
│   │   ├── main.jsx
│   │   └── index.css
│   ├── package.json
│   └── README.md
├── docs/                        # architecture, api, methodology, evaluation
├── uploads/                     # cleaned text + metadata (gitignored)
├── outputs/                     # generated summaries (gitignored)
├── .env.example
└── README.md
```

— End of Report —